

PHASE 4 · MACHINE LEARNING · DAY 21-32

Employee Salary Prediction

Machine Learning · Regression

Scikit-Learn Pipelines

250,000-Record Dataset

CP

Chandra Prakash Choudhary

Data Science & AI Internship

LinkedIn

GitHub

OPTION 5 OF 5 SELECTED

End-to-End ML Pipeline for Global Employee Salary Prediction

Trained on 250,000 employee records spanning 10 features — comparing three regression models and selecting the best performer using R^2 , MAE, and RMSE metrics, then serializing the winning pipeline to disk.

250K
TRAINING RECORDS

3
ML MODELS TESTED

96.3%
BEST R^2 SCORE

Regression

sklearn Pipeline

joblib Persistence

Kaggle Dataset



Internship Brief & Project Scope

CHOSEN PROJECT

Option 5 — Employee Salary Prediction

Build a regression pipeline on the Kaggle Salary Dataset with 250,000 records spanning global job titles, experience, education, industry, company size, location and remote work.

80/20
TRAIN / TEST SPLIT

PHASE 4 TOPICS

- Data Preprocessing
- Feature Engineering
- Regression Modelling
- Model Evaluation

ASSIGNMENT TASKS

- Data Cleaning
- Feature Selection
- 80/20 Train/Test Split
- Train 3 Models
- Compare & Save Best

9
INPUT FEATURES

TARGET VARIABLE

salary (int64)

Continuous numerical output — regression task

Kaggle Dataset — Schema & Salary Statistics

250K
RECORDS

10
COLUMNS

0
NULL VALUES

0
DUPLICATES

FEATURE SCHEMA (10 COLUMNS)

Categorical (6):

job_title · education_level
industry · company_size
location · remote_work

Numerical (3):

experience_years · skills_count
certifications

Target (1): salary

SALARY DISTRIBUTION (\$)



Mean ≈ Median → near-symmetric distribution · Std Dev: \$37,407

Import, Load, Clean & Split

IMPORT LIBRARIES & LOAD DATASET

```
import pandas as pd, numpy as np, joblib
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler
)
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error, r2_score
)

# Load 250,000-row dataset
df = pd.read_csv(
    r"C:\Users\choud\Downloads\"
```

FEATURE SELECTION & 80/20 SPLIT

```
# Remove any duplicate rows (precaution)
df = df.drop_duplicates()

# Separate target from features
target = "salary"
X = df.drop(columns=[target])
y = df[target]

# Auto-detect feature types
categorical_features = X.select_dtypes(
    include=["object", "category"]
).columns.tolist()
# ['job_title', 'education_level', 'industry',
#  'company_size', 'location', 'remote_work']

numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
# ['experience_years', 'skills_count',
```

Sklearn Pipeline — Two-Branch Preprocessor

FULL PIPELINE CODE

```
# — Numerical Branch —————
numeric_pipeline = Pipeline(steps=[
    # Fill missing values with column mean
    ("imputer", SimpleImputer(strategy="mean")),
    # Normalise: mean=0, std=1
    ("scaler", StandardScaler())
])

# — Categorical Branch —————
categorical_pipeline = Pipeline(steps=[
    # Fill missing with most common value
    ("imputer", SimpleImputer(
        strategy="most_frequent")),
    # Convert strings → binary columns
    ("onehot", OneHotEncoder(
        handle_unknown="ignore"))
])

# — Merge Both Branches —————
```

① NUMERICAL BRANCH (3 FEATURES)

1

SimpleImputer — mean

Replaces any NaN with the column arithmetic mean

2

StandardScaler

Normalises each feature: $\mu = 0$, $\sigma = 1$

② CATEGORICAL BRANCH (6 FEATURES)

1

SimpleImputer — most_frequent

Fills missing strings with the modal category

2

OneHotEncoder — handle_unknown="ignore"

Expands each string column into binary indicator columns

③ COLUMNTRANSFORMER MERGES BOTH

Both branches are fused into a single feature matrix, then passed to the regressor. Pipeline ensures no data leakage between train and test sets.

Three Regression Algorithms Under Comparison

01**Linear Regression****HOW IT WORKS**

Fits the best-fit hyperplane across all encoded features by minimizing sum of squared residuals. Fastest to train, highly interpretable.

```
LinearRegression()
```

Parametric

Fast

Interpretable

02**Decision Tree Regressor****HOW IT WORKS**

Recursively partitions data on feature thresholds. Can model non-linear patterns. Depth-limited to max_depth=15 to prevent overfitting.

```
DecisionTreeRegressor(  
    max_depth=15,  
    random_state=42  
)
```

Non-Linear

Tree-Based

03**Random Forest Regressor****HOW IT WORKS**

Ensemble of 30 decision trees with bagging. Reduces variance by averaging predictions. Uses all CPU cores (n_jobs=-1).

```
RandomForestRegressor(  
    n_estimators=30,  
    max_depth=15,  
    n_jobs=-1  
)
```

Ensemble

Robust

Training, Predicting & Evaluating All 3 Models

TRAINING & EVALUATION LOOP

```
results = []
trained_models = {}

for model_name, model in models.items():
    print(f"\nTraining {model_name}...")

    # Build end-to-end pipeline
    model_pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("regressor", model)
    ])

    # Train on 200,000 rows
    model_pipeline.fit(X_train, y_train)

    # Predict on 50,000 unseen rows
    y_pred = model_pipeline.predict(X_test)

    # Compute three evaluation metrics
```

EVALUATION METRICS EXPLAINED

MAE — Mean Absolute Error ↓

Average salary error in dollars. Easy to interpret. Less sensitive to extreme outliers.

RMSE — Root Mean Squared Error ↓

Penalises large errors heavily. Reveals costly outlier predictions.

R² — Coefficient of Determination ↑

Variance explained. 1.0 = perfect · >0.95 = excellent for salary regression.

ACTUAL CONSOLE OUTPUT

Training Linear Regression...

MAE: 5436.10 R²: 0.9635

Training Decision Tree...

MAE: 8689.50 R²: 0.9132

Training Random Forest...

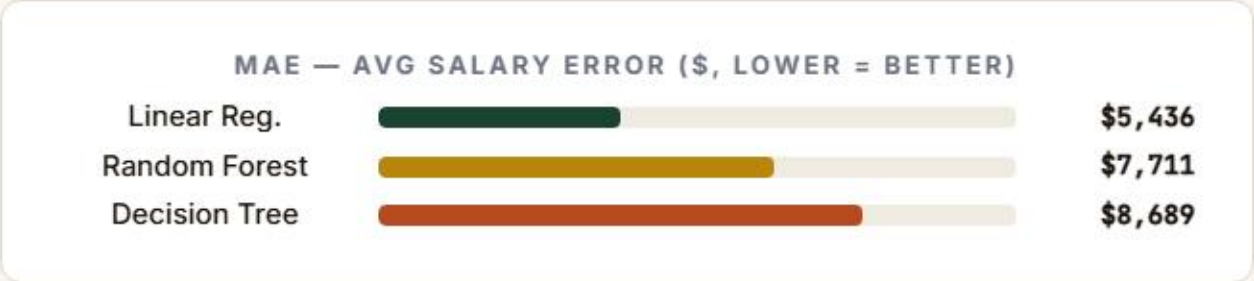
MAE: 7710.94 R²: 0.9307

Model Performance Leaderboard

RANK	MODEL	MAE (\$)	RMSE (\$)	R ²
1	Linear Regression	5,436	7,126	0.9635
2	Random Forest	7,711	9,813	0.9307
3	Decision Tree	8,689	10,983	0.9132

WHY LINEAR REGRESSION WON

After OneHotEncoding, categorical columns expand into hundreds of binary columns. In this high-dimensional sparse space, salary shows a near-perfect linear relationship with experience, skills, and encoded job/location features. Linear regression thrives here.



96.35%

BEST R² SCORE ACHIEVED

Linear Regression · Selected ✓

Selecting & Serialising the Best Model

96.3%
R² SCORE

🏆 **Linear Regression**
Saved as .pkl pipeline

WHAT JOBLIB SAVES

- Fitted ColumnTransformer (encoders + scalers)
- Trained LinearRegression weight vector
- Full Pipeline wrapper for one-call inference
- Deploy-ready for a salary prediction API

BEST MODEL SELECTION & SAVE CODE

```
# Sort results DataFrame by R2 (desc)
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(
    by="R2 Score", ascending=False
)

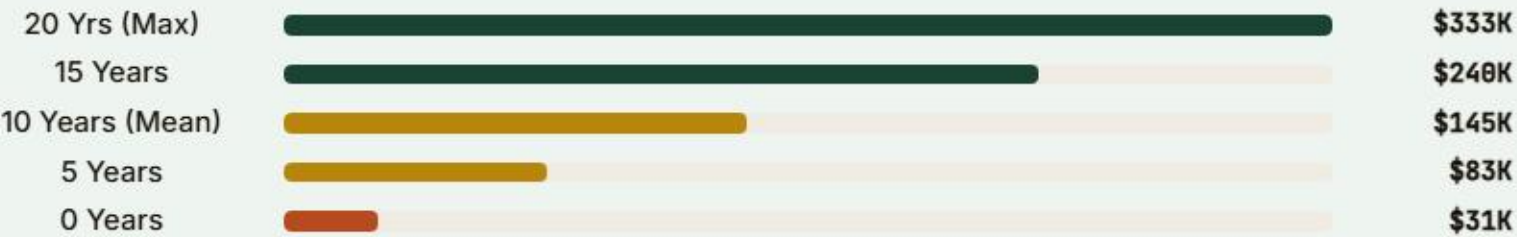
print("Model Comparison:")
print(results_df)
```

#	Model	MAE	RMSE	R2 Score	
#	Linear Reg.	5,436	7,126	0.9635	← best
#	Random Forest	7,711	9,813	0.9307	
#	Decision Tree	8,689	10,983	0.9132	

```
# Select top model automatically
best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]
```

Salary Drivers — What the Data Reveals

EXPERIENCE VS SALARY



Experience shows the strongest linear correlation to salary across all categories.

EDUCATION LEVEL PREMIUM

PhD holders command a salary premium over Bachelor's graduates. However, years of experience shows a stronger linear correlation to salary than education level alone — making it the dominant predictor.

REMOTE WORK PREMIUM

Employees in fully remote positions across high-cost countries (Sweden, Canada, USA) commanded salary premiums of up to 20% versus equivalent hybrid or on-site roles — reflecting the global talent market.

COMPANY SIZE EFFECT

Large enterprises pay significantly higher average salaries than small companies, with the widest gaps in Technology and Finance sectors. Medium companies sit midway on both salary and job security dimensions.

SKILLS & CERTIFICATIONS

Each additional certification adds measurable salary uplift. The model captured that skills_count and certifications act as multiplicative salary boosters on top of base experience.

Machine Learning Competencies Developed

① SKLEARN PIPELINE ENGINEERING

Built production-grade sklearn.Pipeline combining imputation, scaling and encoding — a reusable, data-leak-proof workflow that applies identical transforms to train and test data.

③ MULTI-MODEL BENCHMARKING

Systematically trained and evaluated 3 regression algorithms on 50,000 unseen test samples using MAE, RMSE, and R^2 to select the statistically best performer — Linear Regression with $R^2 = 0.9635$.

② FEATURE ENGINEERING AT SCALE

Applied OneHotEncoder to 6 high-cardinality categorical columns and StandardScaler to 3 numeric columns inside a parallel ColumnTransformer step, handling 250,000 rows.

④ MODEL PERSISTENCE & DEPLOYMENT READINESS

Used joblib.dump() to serialize the full fitted pipeline to a .pkl binary file. The saved object includes fitted encoders, scalers, and model weights — ready for a real-time salary prediction API.

Employee Salary Prediction

BEST MODEL
Linear Regression

R² SCORE
96.35%

TRAINING RECORDS
200,000

MEAN SALARY ERROR
\$5,436

PHASE 4 COMPLETE

Thank You.

Questions, feedback, and discussion are welcome. This project demonstrates a complete ML workflow — from raw Kaggle data to a serialised, deployment-ready regression pipeline.

sklearn Pipelines

3 Regressors Tested

R² = 0.9635

joblib .pkl

250K Records

CP

Chandra Prakash Choudhary

[LinkedIn](#)

[GitHub](#)

